

Practical 5: Dockerize a Flask Application

Aim

To create a Docker image for a Flask application using a Dockerfile.

Objective

- Create a Flask application.
 - Create a requirements.txt file.
 - Write a Dockerfile.
 - Build a Docker image.
 - Verify the Docker image.
-

Software Requirements

Software	Version
Windows 10/11	Any
Python	3.12+
Docker Desktop	Latest
VS Code	Latest
Flask	Latest

A. Write a Dockerfile for your Flask/FastAPI application and build a Docker image

Step 1: Install Docker Desktop

1. Open your browser.
 2. Visit <https://www.docker.com/products/docker-desktop/>
 3. Download Docker Desktop for Windows.
 4. Install it.
 5. Restart your computer if prompted.
 6. Open Docker Desktop.
 7. Wait until the Docker Engine status shows **Running**.
-

Step 2: Verify Docker Installation

Open **Command Prompt** or **PowerShell**.

Type:

```
docker --version
```

Example Output

Docker version 28.3.2, build xxxx

Now check

```
docker info
```

If Docker is running, lots of system information will be displayed.

Step 3: Create Project Folder

Open Command Prompt.

```
mkdir Flask_Project
```

Go inside the folder.

```
cd Flask_Project
```

Step 4: Open the Folder in VS Code

Type

```
code .
```

VS Code opens.

Step 5: Create Flask Application

Create a new file

```
app.py
```

Write the following code.

```
from flask import Flask
```

```

app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome to Dockerized Flask Application"

@app.route("/about")
def about():
    return "Docker Practical"

if __name__ == "__main__":
    app.run(host="0.0.0.0",port=5000)

```

Explanation

Statement	Purpose
Flask()	Creates the Flask application
@app.route("/")	Defines the Home page
@app.route("/about")	Defines another page
host="0.0.0.0"	Allows Docker to access the application
port=5000	Flask runs on port 5000

Step 6: Create Virtual Environment (Optional but Recommended)

python -m venv venv

Activate

Windows

venv\Scripts\activate

You should see

(venv)

at the beginning of the command prompt.

Step 7: Install Flask

pip install flask

Wait until installation completes.

Step 8: Test the Application

Run

```
python app.py
```

Output

Running on

```
http://127.0.0.1:5000
```

Open browser

```
http://localhost:5000
```

Output

Welcome to Dockerized Flask Application

Also check

```
http://localhost:5000/about
```

Output

Docker Practical

Step 9: Create requirements.txt

This file contains all Python packages required by the application.

Method 1 (Recommended)

```
pip freeze > requirements.txt
```

Open the file.

It may look like

```
blinker==1.9.0
```

```
click==8.2.1
Flask==3.1.0
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.2
Werkzeug==3.1.3
```

Method 2

Create

requirements.txt

Write

Flask

Method 1: Automatically Create `requirements.txt` (Recommended)

Step 1: Open VS Code

Open your Flask project.

Example:

```
Flask_Project
```

Step 2: Open the Terminal

Click

Terminal

↓

New Terminal

You should see something like:

```
PS C:\Users\YourName\Documents\Flask_Project>
```

Step 3: Activate the Virtual Environment (if you created one)

If your virtual environment is named `venv`, type:

```
venv\Scripts\activate
```

After activation, your prompt will look similar to:

```
(venv) PS C:\Users\YourName\Documents\Flask_Project>
```

If you did **not** create a virtual environment, you can skip this step.

Step 4: Check That Flask Is Installed

Type:

```
pip list
```

You should see something like:

Package	Version
Flask	3.1.0
click	8.2.1
Werkzeug	3.1.3
Jinja2	3.1.6

If Flask is **not** listed, install it:

```
pip install flask
```

Step 5: Generate `requirements.txt`

Type:

```
pip freeze > requirements.txt
```

Press **Enter**.

Nothing will appear on the screen, but the file has been created.

Step 6: Verify the File

In VS Code, you should now see:

```
Flask_Project
├── app.py
├── requirements.txt
└── Dockerfile
```

Double-click **requirements.txt**.

It may contain:

```
blinker==1.9.0
click==8.2.1
Flask==3.1.0
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.2
Werkzeug==3.1.3
```

This is exactly what Docker will use to install the required packages.

Step 10: Create Dockerfile

Create a file named

Dockerfile

Important: Do **not** give it an extension like .txt. The filename must be exactly Dockerfile.

Paste the following:

```
# Step 1: Use Python base image
FROM python:3.12-slim
```

```
# Step 2: Set working directory
WORKDIR /app
```

```
# Step 3: Copy requirements file
COPY requirements.txt .
```

```
# Step 4: Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Step 5: Copy all project files
COPY . .
```

```
# Step 6: Open Flask port
EXPOSE 5000
```

```
# Step 7: Start the application
CMD ["python", "app.py"]
```

Understanding Each Dockerfile Command

1. FROM

```
FROM python:3.12-slim
```

Purpose

- Downloads the official Python image.
- This becomes the base operating system for your container.

2. WORKDIR

WORKDIR /app

Purpose

Creates a folder

/app

inside Docker.

All files are copied here.

3. COPY

COPY requirements.txt .

Purpose

Copies

requirements.txt

from your computer into the Docker container.

4. RUN

RUN pip install --no-cache-dir -r requirements.txt

Purpose

Installs Flask and any other required Python packages inside the Docker image.

5. COPY

COPY . .

Purpose

Copies your project files (app.py, etc.) into the container.

6. EXPOSE

EXPOSE 5000

Purpose

Tells Docker that the application listens on port **5000**.

7. CMD

CMD ["python","app.py"]

Purpose

Specifies the command Docker runs automatically when the container starts.

Step 11: Verify Project Structure

Your folder should look like this:

```
Flask_Project/
├── app.py
├── Dockerfile
├── requirements.txt
└── venv/    (optional)
```

Step 12: Open Terminal

Make sure you are inside

Flask_Project

Verify by running:

dir

You should see

app.py

Dockerfile

requirements.txt

Step 13: Build Docker Image

Run

```
docker build -t flask-app .
```

Explanation

Part	Meaning
docker	Docker command
build	Build an image
-t	Assign a tag (name)
flask-app	Name of the image
.	Current directory contains the Dockerfile

Docker will:

1. Download the Python image (if not already available).
2. Copy your files.
3. Install Flask.
4. Create the Docker image.

Example output:

Successfully built 8b1234567890

Successfully tagged flask-app:latest

Step 14: Verify the Docker Image

Run:

```
docker images
```

Example output:

Repository	Tag	Image ID	Size
flask-app	latest	8b1234567890	170 MB

This confirms that the image has been built successfully.

Expected Result

- Docker Desktop is installed and running.
- Flask application works locally.
- requirements.txt is created.
- Dockerfile is created.
- Docker image is successfully built.
- The image appears in the output of docker images.

Common Errors and Solutions

Error	Possible Cause	Solution
docker: command not found	Docker Desktop is not installed or not running	Install Docker Desktop and ensure it is running
Dockerfile not found	Incorrect filename or wrong folder	Name the file exactly Dockerfile and run the command from the project folder
No module named flask	Flask not installed or missing from requirements.txt	Install Flask (pip install flask) and regenerate requirements.txt
Build fails while installing packages	Incorrect package name or internet issue	Check requirements.txt and try the build again

This completes **Part (a): Write a Dockerfile for your Flask/FastAPI application and build a Docker image**. The next practical would be **(b) Run the Docker container and test the application**, followed by **(c) Push the Docker image to Docker Hub**.